

Batch: B1 Roll No.: 16010122158

Experiment No.: 3

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

Title: Study of Umbrello Unified Modelling Language tool Or Lucidchart: flowcharts and diagram drawing tool

Aim: To learn and understand the way of creating various UML diagrams for requirement analysis

CO: Analyze the software requirements and Model the defined problem with the help of UML diagram.

Books/ Journals/ Websites referred:

1. Roger Pressman, "Software Engineering", sixth edition, Tata McGraw Hill.
 2. System Analysis & Design by Satzinger, Jackson and Burd, Cengage Learning, 2007
 3. System Analysis and Design Methods by Jeffery I. Whitten, Lonnie D Bentley, McGraw Hill, 7th edition.
 4. System Analysis and Design by Alan Dennis, Barbara H. Wixom, Roberta M. Roth, Wiley India 4th edition
 5. http://en.wikipedia.org/wiki/Software_requirements_specification
 6. http://en.wikipedia.org/wiki/Use_case
-

Pre Lab/ Prior Concepts:

In software and systems engineering, a **use case** is a list of steps, typically defining interactions between a role (known in Unified Modeling Language (UML) as an "actor") and a system, to achieve a goal. The actor can be a human or an external system.

In systems engineering, use cases are used at a higher level than within software engineering, often representing missions or stakeholder goals. The detailed requirements may then be captured in Systems Modeling Language (SysML) or as contractual statements.

As an important requirement technique, use cases have been widely used in modern software engineering over the last two decades. Use case driven development is a key characteristic of process models and frameworks like Unified Process (UP), Rational Unified Process (RUP), Oracle Unified Method (OUM), etc. With its iterative and evolutionary nature, use case is also a good fit for agile development.

A sequence diagram is a graphical view of a scenario that shows object interaction in a time-based sequence of what happens first, what happens next.

Sequence diagrams establish the roles of objects and help provide essential information to determine class responsibilities and interfaces.

In UML, class diagrams are one of six types of structural diagram. Class diagrams are fundamental to the object modelling process and model the static structure of a system. Depending on the complexity of a system, you can use a single class diagram to model an entire system, or you can use several class diagrams to model the components of a system.

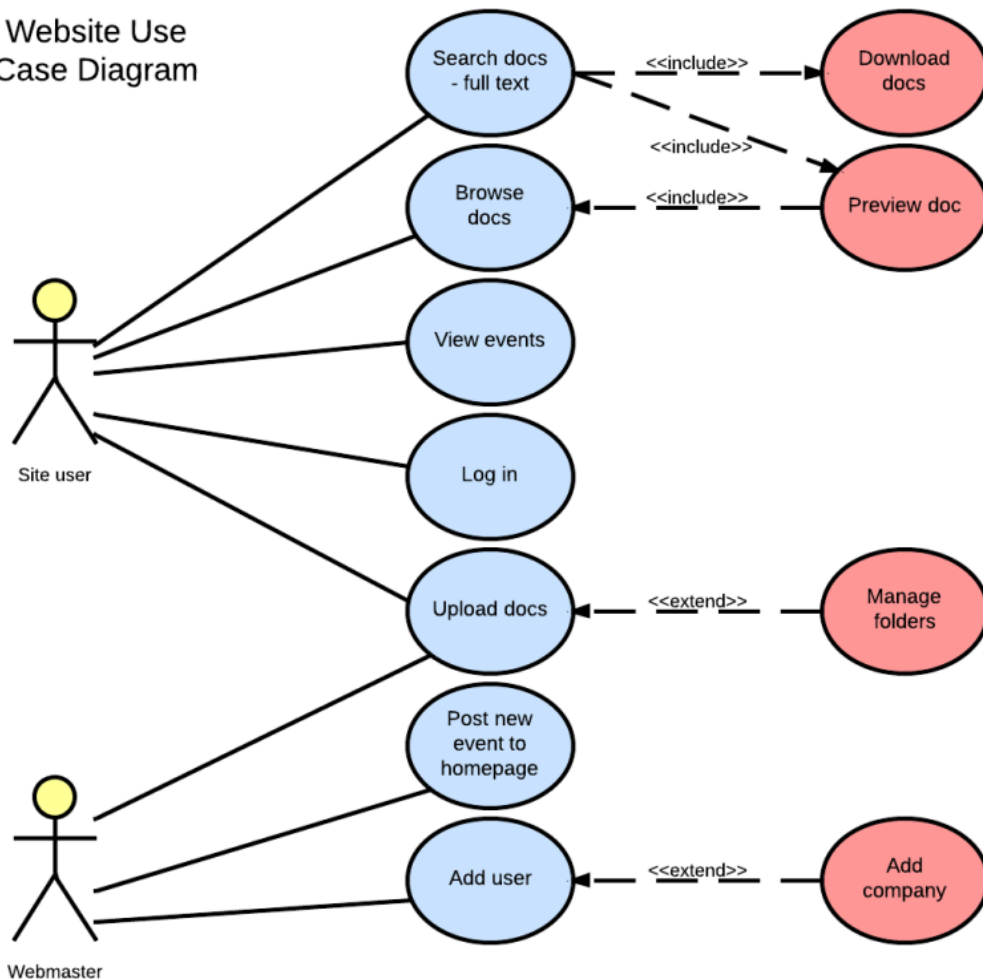
Class diagrams are the blueprints of your system or subsystem. You can use class diagrams to model the objects that make up the system, to display the relationships between the objects, and to describe what those objects do and the services that they provide.

In its basic form, an activity diagram is a simple and intuitive illustration of what happens in a workflow, what activities can be done in parallel, and whether there are alternative paths through the workflow. Activity diagrams as defined in the Unified Modeling Language are derived from various techniques to visually illustrate workflows. Activity diagrams are used to visualize the workflow of a business use case. A complete workflow description will have a basic flow, and one or several alternative flows. This workflow has a structure that we can define textually, using informal if, if-then-else, or does-until statements of various kinds. For a simple workflow with a simple structure such textual definitions may be quite sufficient, but in the case of more complex structures, activity diagrams help to clarify and make more apparent what the workflow is. Historically, activity diagramming techniques have mostly been used in the business process modeling domain, but this article will also briefly discuss how you can use it in the system modeling domain.

Requirement Modeling:

1. Use Case

Website Use
Case Diagram



Description of the use case diagram:-

Actors:

1. **Site User** (stick figure at the top left):
 - This actor represents a general user of the website who can perform various tasks like searching, browsing, viewing events, logging in, and uploading documents.
2. **Webmaster** (stick figure at the bottom left):
 - This actor represents a website administrator with additional privileges, such as adding users and posting new events.

Use Cases (Ellipses):

1. For Site User:

- **Search docs - full text:** The user can search for documents by their full text.
- **Browse docs:** The user can browse through available documents.
- **View events:** The user can view upcoming or past events.
- **Log in:** The user can log into the website to access more features.
- **Upload docs:** The user can upload documents to the system.

2. For Webmaster:

- **Post new event to homepage:** The webmaster can add new events to the website homepage.
- **Add user:** The webmaster can add new users to the system.

Relationships (Arrows):

1. Include Relationships (Dashed arrow with label):

- **Search docs - full text** includes **Download docs:** When searching for documents, the user can also download the documents as a part of the process.
- **Browse docs** includes **Preview doc:** When browsing through documents, the user can preview them before taking further actions.

2. Extend Relationships (Dashed arrow with label):

- **Upload docs** extends **Manage folders:** When uploading documents, the user has the option to manage folders (e.g., creating, renaming, or deleting folders).
- **Add user** extends **Add company:** When adding users, the webmaster can also add a company, implying that the user might be associated with an organization.

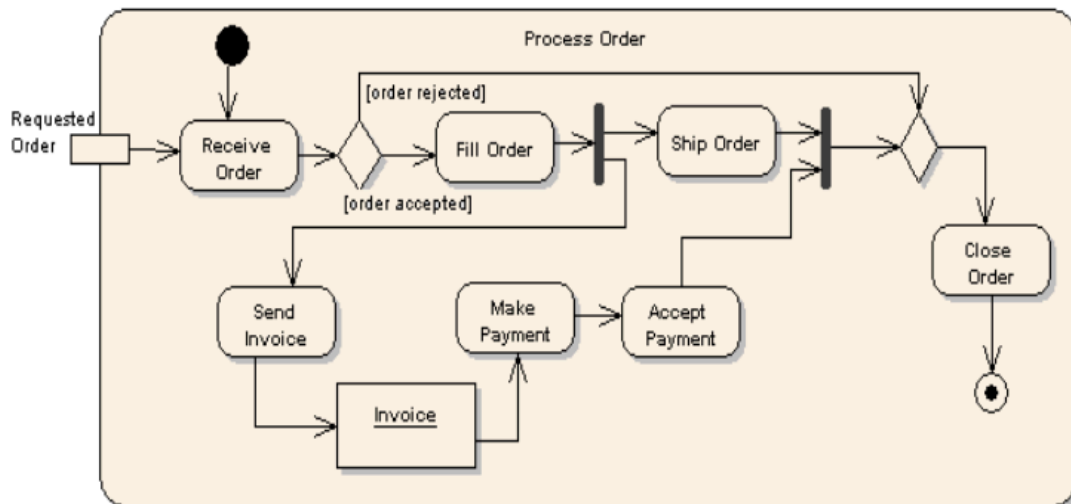
Key Points:

- **Site User** has standard access to the website features such as searching, browsing, uploading documents, and viewing events.
- **Webmaster** has administrative privileges, including managing users and posting events.

- **Include Relationships** indicate mandatory functionality that is part of the broader tasks (e.g., downloading or previewing documents when searching or browsing).
- **Extend Relationships** indicate optional functionalities that can be performed under certain conditions (e.g., managing folders when uploading documents, adding a company when adding a user).

This diagram effectively showcases the different levels of access and actions available to both regular users and administrators of the website.

2. Activity Diagram



Description of the Activity Diagram:

Start of Process:

The process begins with the Requested Order coming into the system. This is indicated by an initial filled circle, which represents the starting point of the activity.

Receive Order:

The first activity is Receive Order. This is where the system receives the order that has been requested. It's represented by a rounded rectangle.

Decision Node (Order Acceptance):

After receiving the order, there is a decision point (diamond shape) where the system checks if the order is accepted or rejected.

Order Rejected: If the order is rejected, the process ends at this decision point (no further steps are shown for this branch).

Order Accepted: If the order is accepted, the process continues to the next steps.

Fill Order:

If the order is accepted, the next activity is Fill Order. This is where the system prepares the order for shipping.

Ship Order:

After the order is filled, the next step is Ship Order, where the order is sent to the customer. There is a synchronization bar (a thick line) before and after this activity, indicating that this step must be completed before moving on to subsequent activities.

Decision Node (Order Shipment Acceptance):

After shipping, another decision point is reached where the system checks if the shipment is accepted.

If the shipment is accepted, the process continues to Close Order.

The diagram doesn't specify what happens if the shipment is not accepted, indicating the focus is on successful shipment handling.

Send Invoice:

Parallel to the shipping process, another activity is Send Invoice, where the system generates and sends an invoice for the order.

Make Payment:

The next activity is Make Payment, where the customer is required to make the payment.

Accept Payment:

After the payment is made, the next step is Accept Payment, where the system processes the payment.

Invoice and Payment Handling:

The invoice and payment activities are depicted as happening in parallel with the order filling and shipping process, which is why there are synchronization bars. This parallel processing ensures that the financial aspects of the order are handled alongside the physical delivery.

Close Order:

After the order is shipped and payment is accepted, the final step is Close Order, where the system closes the order, marking the end of the process.

End of Process:

The process concludes with a filled circle inside another circle, representing the end of the activity flow.

Significance of Activity Diagram

An Activity Diagram is a crucial tool in the field of software engineering and systems modeling. Its significance lies in its ability to visually represent the flow of activities or actions within a system or process. Here are some key points that highlight the importance of activity diagrams:

1. Visual Representation of Workflows:

- Activity diagrams provide a clear, visual depiction of the sequence of activities and decisions in a process. This helps stakeholders understand how a system operates, including the flow of control from one activity to another.

2. Modeling Concurrent Processes:

- They are particularly useful for modeling parallel or concurrent processes. The use of synchronization bars in activity diagrams can illustrate how different activities can happen simultaneously, or how processes must synchronize before continuing.

3. Decision Making and Conditional Flows:

- Activity diagrams show decision points (branching) within a process. This is essential for representing scenarios where different outcomes lead to different paths in the workflow, enabling clearer communication of business logic and process flows.

4. System Behavior and Use Case Implementation:

- They are often used to model the implementation of use cases. By detailing the sequence of activities involved in a use case, an activity diagram provides a blueprint for how the system should behave in various scenarios.

5. Clarifying Complex Processes:

- Complex processes with multiple steps and dependencies can be challenging to describe textually. Activity diagrams simplify this by breaking down the process into a series of actions and decisions, making it easier to understand and analyze.

6. Communication Among Stakeholders:

- These diagrams serve as a common language between developers, business analysts, and non-technical stakeholders. They bridge the gap between technical and non-technical team members, ensuring everyone has a shared understanding of the process.

7. Identifying Bottlenecks and Optimizing Processes:

- By visualizing the flow of activities, stakeholders can identify potential bottlenecks or inefficiencies in a process. This insight can lead to process improvements and optimizations, ensuring the system operates more effectively.

8. Documentation and Compliance:

- Activity diagrams are often used as part of system documentation. They provide a formal record of the workflow and processes within a system, which can be critical for compliance, auditing, and future reference.

9. Supporting Test Case Design:

- The detailed flow of actions in an activity diagram helps in designing test cases. Each path through the diagram can correspond to a test case, ensuring that all scenarios are covered during testing.

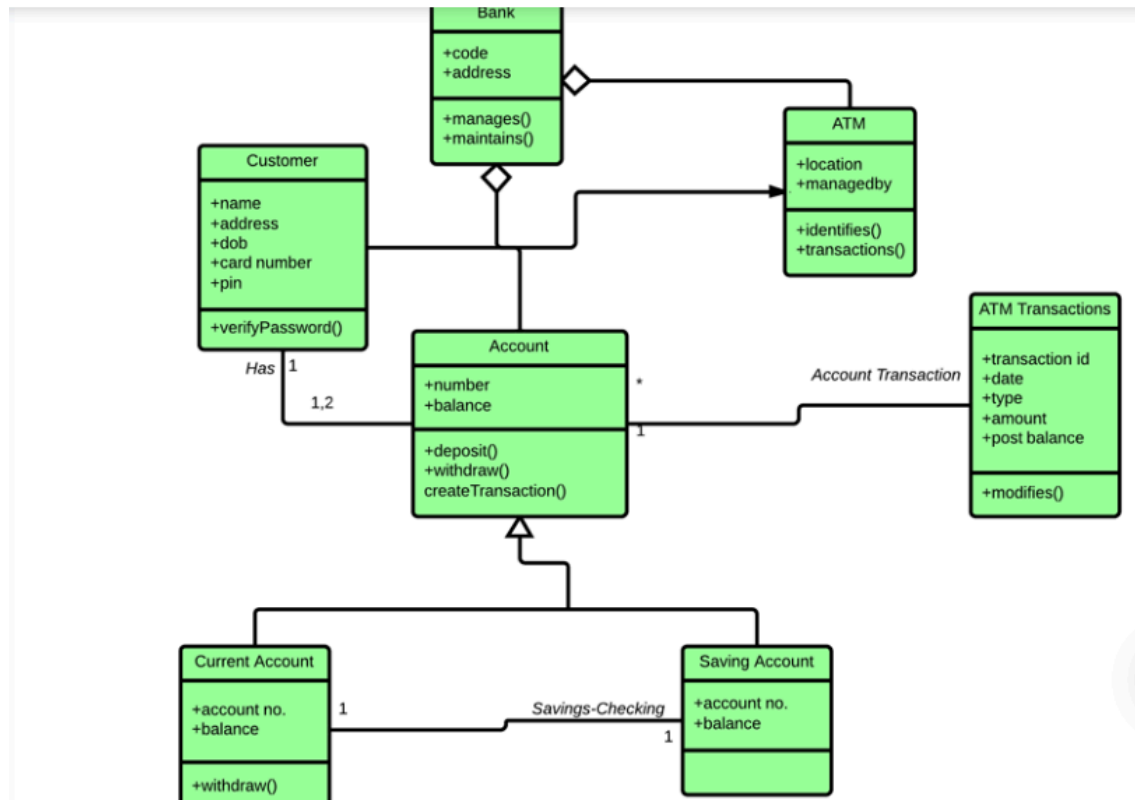
10. Facilitating Automation and Workflow Management:

- In systems that involve workflow management or business process automation, activity diagrams are instrumental in defining the exact sequence of steps that need to be automated. They can serve as input for developing workflow engines or automation scripts.

3. Class Diagram

Class diagram for an ATM system:

ATMs are deceptively simple: although customers only need to press a few buttons to receive cash, there are many layers of security that a safe and effective ATM must pass through to prevent fraud and provide value for banking customers. The various human and inanimate parts of an ATM system are illustrated by this easy-to-read diagram—every class has its title, and the attributes are listed beneath.



Classes and their Attributes/Methods:

1. Bank

- Attributes: **code**, **address**
- Methods: **manages()**, **maintains()**
- Significance: Represents the bank that owns and manages the ATMs and accounts. The bank has a unique code and an address.

2. ATM

- Attributes: **location**, **managedby**
- Methods: **identifies()**, **transactions()**
- Significance: Represents the ATM machines managed by the bank. Each ATM has a specific location and is managed by a particular branch of the bank.

3. Customer

- Attributes: **name**, **address**, **dob** (date of birth), **card number**, **pin**
- Methods: **verifyPassword()**

- Significance: Represents a bank customer who holds an account and can access ATM services using a card and PIN. The customer class has personal details necessary for account verification and transactions.

4. Account

- Attributes: **number**, **balance**
- Methods: **deposit()**, **withdraw()**, **createTransaction()**
- Significance: Represents a generic account in the bank, which could be of different types like current or savings. The account class holds the account number and balance and allows deposit, withdrawal, and transaction creation functionalities.

5. Current Account

- Attributes: **account no.**, **balance**
- Methods: **withdraw()**
- Significance: A specialized type of account (inherits from Account) meant for day-to-day business transactions. It includes all functionalities of a general account but typically focuses on withdrawals.

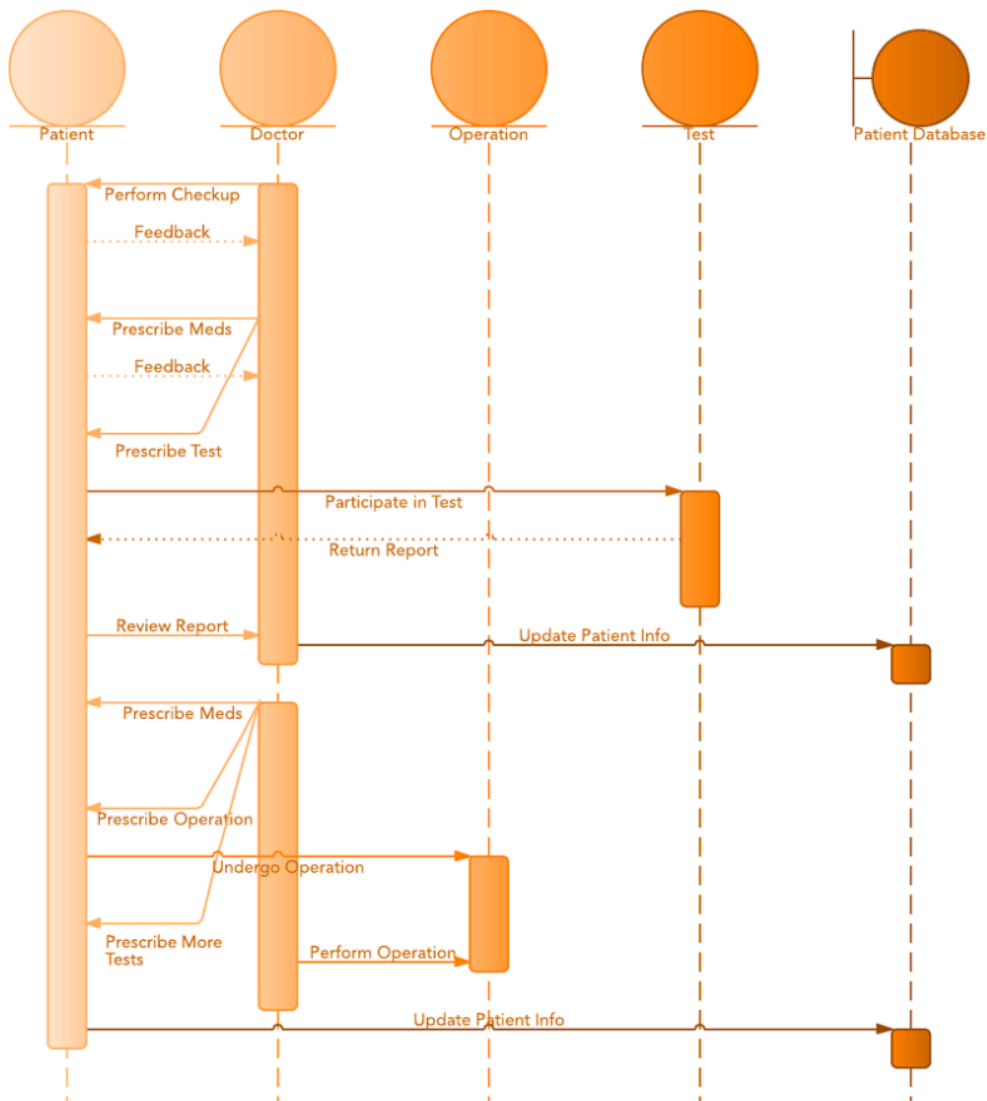
6. Saving Account

- Attributes: **account no.**, **balance**
- Methods: None specified
- Significance: Another specialization of the Account class, representing savings accounts that earn interest. They are mainly used for saving money and usually limit withdrawals.

7. ATM Transactions

- Attributes: **transaction id**, **date**, **type**, **amount**, **post balance**
- Methods: **modifies()**
- Significance: Represents the transactions carried out via ATMs. It records the details of each transaction, such as the transaction type (deposit or withdrawal), date, amount, and balance after the transaction.

4. Sequence



Description of given sequence diagram:-

Actors and Objects:

1. **Patient:** Represents the individual receiving medical care.
2. **Doctor:** Represents the healthcare provider responsible for diagnosing and prescribing treatments.
3. **Operation:** Refers to a medical procedure that the patient might undergo.
4. **Test:** Represents a medical test or diagnostic procedure prescribed by the doctor.

5. **Patient Database:** The system or database where patient information is stored and updated.

Interactions (Messages between Entities):

1. **Perform Checkup:** The interaction starts with the patient visiting the doctor, who performs a checkup.
2. **Feedback:** After the checkup, the doctor provides feedback to the patient, which can be general advice or next steps in the treatment.
3. **Prescribe Meds:** The doctor prescribes medication to the patient as part of the treatment plan.
4. **Feedback (After Meds):** The doctor provides feedback to the patient, possibly regarding the medication's effects or usage instructions.
5. **Prescribe Test:** The doctor prescribes a medical test that the patient needs to undergo.
6. **Participate in Test:** The patient undergoes the prescribed test.
7. **Return Report:** After the test is completed, the results (report) are returned to the doctor for review.
8. **Review Report:** The doctor reviews the test report to assess the patient's condition and determine further actions.
9. **Update Patient Info:** The patient database is updated with the new information from the test results.
10. **Prescribe Operation:** If the test results suggest, the doctor prescribes a surgical operation for the patient.
11. **Prescribe More Tests:** The doctor may prescribe additional tests if needed.
12. **Undergo Operation:** The patient undergoes the prescribed surgical operation.
13. **Perform Operation:** The operation is performed, likely by a surgeon or a medical team associated with the operation entity.
14. **Update Patient Info (After Operation):** After the operation, the patient's information is updated in the patient database.

Sequence Flow:

- The sequence starts with the patient visiting the doctor for a checkup, leading to various possible actions such as prescribing medication, tests, or surgery.

- Tests and operations involve interactions with external entities, such as laboratories (for tests) and surgical teams (for operations).
- Information is consistently updated in the patient database after tests and operations to maintain accurate and current patient records.

Key Points:

- **Lifelines (Vertical dashed lines):** Represent the existence of each entity over time as the interaction progresses.
- **Synchronous Messages (Solid arrows):** Indicate direct communication between entities, like performing a test or prescribing medication.
- **Asynchronous Messages (Dashed arrows):** Indicate feedback or report submission actions that do not require an immediate response from the recipient.
- **Execution Occurrences (Rectangles on lifelines):** Represent periods during which specific operations or activities are being performed (e.g., "Perform Operation").
- **Patient Database Updates:** The diagram emphasizes updating patient information after significant actions like tests and operations.

This sequence diagram effectively models a series of healthcare-related processes, showing how various actions and communications occur between the patient, doctor, medical procedures, and the patient database.

Conclusion:

The study of UML tools like Umbrello and Lucidchart helps in understanding the creation and analysis of various UML diagrams, which are essential for requirement modeling and system design in software engineering. These tools enable the visualization of different aspects of a system, such as use cases, activity flows, class structures, and sequence of operations, providing clarity in system behavior, design, and implementation. By using UML diagrams, stakeholders can better communicate, analyze, and optimize the software development process, ensuring that the system meets its intended requirements.

Post Lab Descriptive Questions:

1. Use Cases in the Software Development Life Cycle (SDLC)

Use cases play a critical role in several phases of the Software Development Life Cycle (SDLC):

- **Requirements Gathering and Analysis:**
 - **Primary Phase:** Use cases are most relevant during the requirements gathering and analysis phases. Here, they help in capturing and documenting functional requirements by describing the interactions between the users (actors) and the system. This ensures that the development team understands what the system is supposed to do from the user's perspective.
- **Design:**
 - **Supporting Phase:** In the design phase, use cases guide the creation of more detailed models like sequence diagrams, class diagrams, and state diagrams. These models help in visualizing how the system will be structured and how it will function internally to meet the use case requirements.
- **Implementation:**
 - **Supporting Phase:** While not directly involved, use cases help ensure that the code being written aligns with the documented requirements. They serve as a reference to ensure that all functionalities are being implemented as intended.
- **Testing:**
 - **Supporting Phase:** Use cases are crucial in the testing phase as they serve as the basis for creating test cases. Each use case can be converted into a test scenario to verify that the system behaves as expected when interacting with users or other systems.
- **Maintenance:**
 - **Supporting Phase:** During maintenance, use cases help in understanding the system's existing functionalities when making modifications or debugging. They serve as a reference point for assessing the impact of changes.

2. Comparison of Sequence Diagram vs. Collaboration Diagram

Sequence Diagram:

- **Description:** A sequence diagram illustrates how objects interact in a particular sequence, focusing on the time order of messages exchanged between objects.
- **Pros:**
 - **Time Sequence Clarity:** It clearly shows the order of interactions, which is beneficial for understanding the sequence of events.
 - **Easy to Trace Process Flow:** Ideal for identifying how processes unfold over time, making it easier to detect bottlenecks or errors in the sequence.
 - **Useful for Identifying Timelines:** It's excellent for understanding and visualizing the timing of interactions, which is crucial for real-time systems.
- **Cons:**
 - **Complexity with Many Objects:** As the number of interacting objects increases, the diagram can become cluttered and difficult to read.
 - **Limited Structural View:** It focuses on the time sequence rather than the relationships and structure of objects, which might be a limitation when analyzing the overall system architecture.

Collaboration Diagram (also known as Communication Diagram):

- **Description:** A collaboration diagram emphasizes the structural organization of objects and their relationships, showing how they communicate with each other through messages.
- **Pros:**
 - **Structural Clarity:** Provides a clear view of the relationships and roles of different objects in the system.
 - **Better for Object Interactions:** Focuses on how objects are linked and interact with each other, which is useful for understanding object roles and responsibilities.

- Handles Complex Interactions Well: Can handle more complex scenarios by emphasizing object collaboration, making it easier to understand the system's structure.
- Cons:
 - Less Focus on Timing: It does not emphasize the time sequence of interactions, which can be a drawback when the order of events is important.
 - Can Become Overwhelming: With many objects and interactions, the diagram can become difficult to follow, especially if the relationships are complex.

Summary of Comparison:

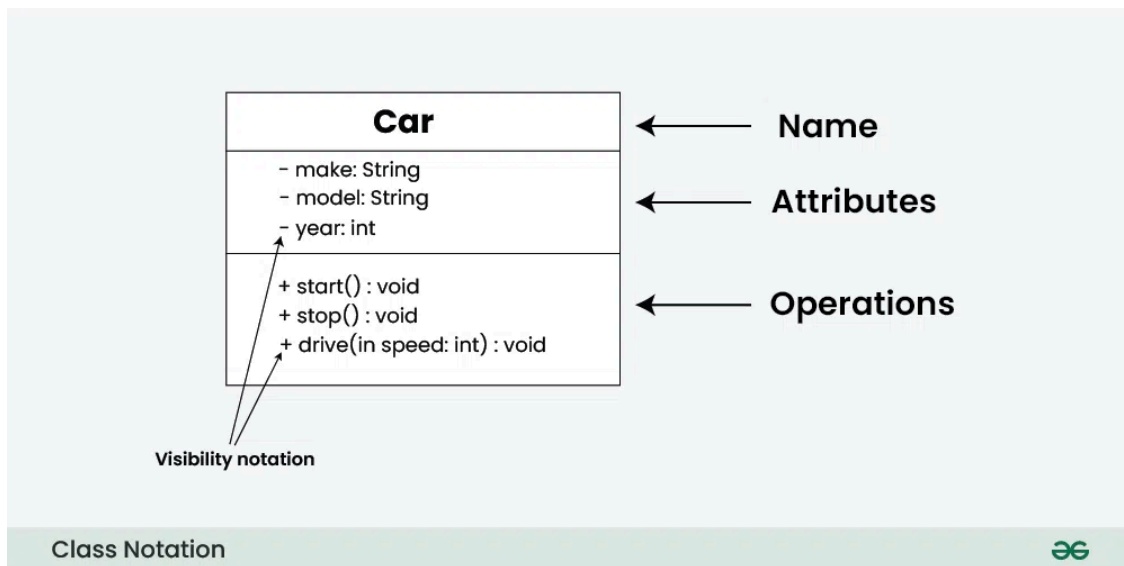
- Sequence Diagram is preferred when the focus is on understanding the time sequence of events in a system. It's especially useful in scenarios where the timing and order of operations are crucial.
- Collaboration Diagram is more effective when the emphasis is on understanding the structural relationships and how objects work together to perform tasks. It's particularly useful for understanding the system's architecture and how different components interact.

Both diagrams complement each other and can be used together to provide a comprehensive understanding of the system from both a temporal and structural perspective.

3. List different notations used in Class diagram with example

UML Class Notation

Class notation is a graphical representation used to depict classes and their relationships in object-oriented modeling.



1. Class Name:

- The name of the class is typically written in the top compartment of the class box and is centered and bold.

2. Attributes:

- Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

3. Methods:

- Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

4. Visibility Notation:

- Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
 - + for public (visible to all classes)
 - - for private (visible only within the class)
 - # for protected (visible to subclasses)
 - ~ for package or default visibility (visible to classes in the same package)

Relationships between classes

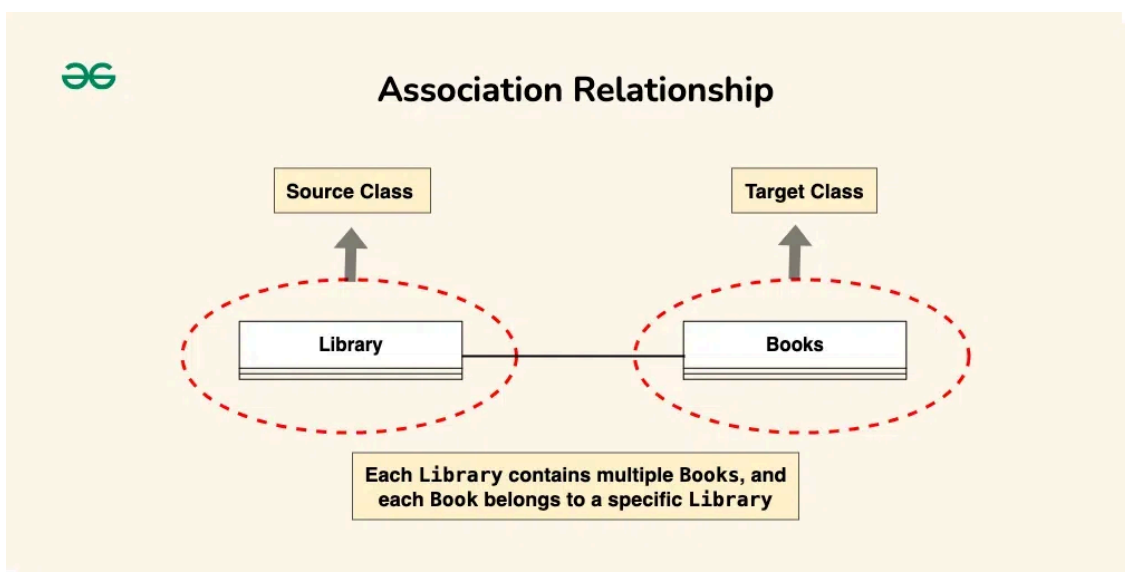
1. Association

An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class. Associations are typically depicted as a solid line connecting the classes, with optional arrows indicating the direction of the relationship.

Let's understand association using an example:

Let's consider a simple system for managing a library. In this system, we have two main entities: Book and Library. Each Library contains multiple Books, and each Book belongs to a specific Library. This relationship between Library and Book represents an association.

The "Library" class can be considered the source class because it contains a reference to multiple instances of the "Book" class. The "Book" class would be considered the target class because it belongs to a specific library.



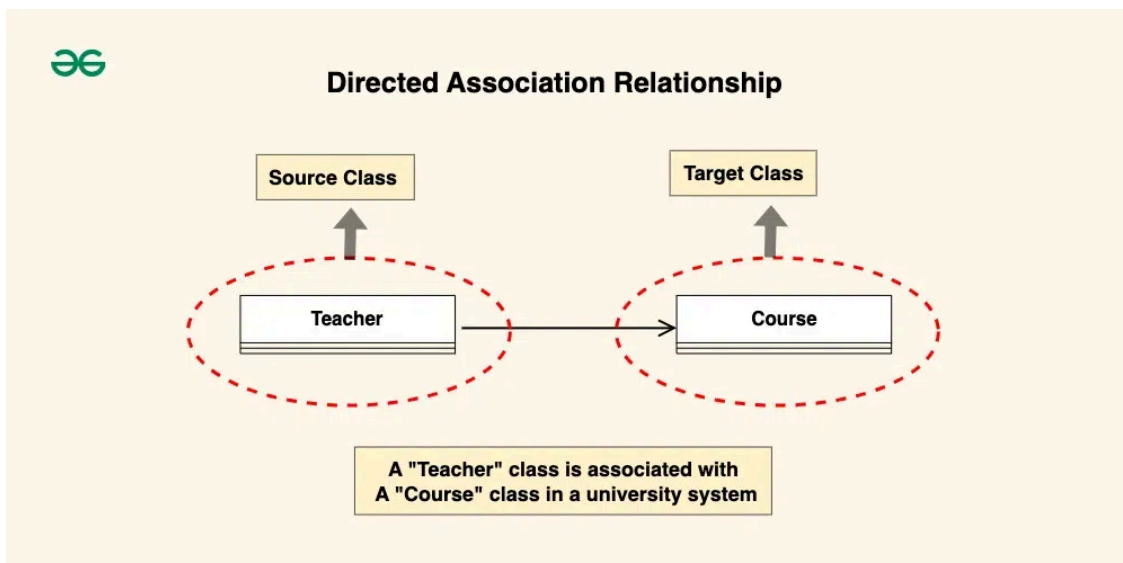
2. Directed Association

A directed association in a UML class diagram represents a relationship between two classes where the association has a direction, indicating that one class is associated with another in a specific way.

- In a directed association, an arrowhead is added to the association line to indicate the direction of the relationship. The arrow points from the class that initiates the association to the class that is being targeted or affected by the association.
- Directed associations are used when the association has a specific flow or directionality, such as indicating which class is responsible for initiating the association or which class has a dependency on another.

Consider a scenario where a “Teacher” class is associated with a “Course” class in a university system. The directed association arrow may point from the “Teacher” class to the “Course” class, indicating that a teacher is associated with or teaches a specific course.

- The source class is the “Teacher” class. The “Teacher” class initiates the association by teaching a specific course.
- The target class is the “Course” class. The “Course” class is affected by the association as it is being taught by a specific teacher.

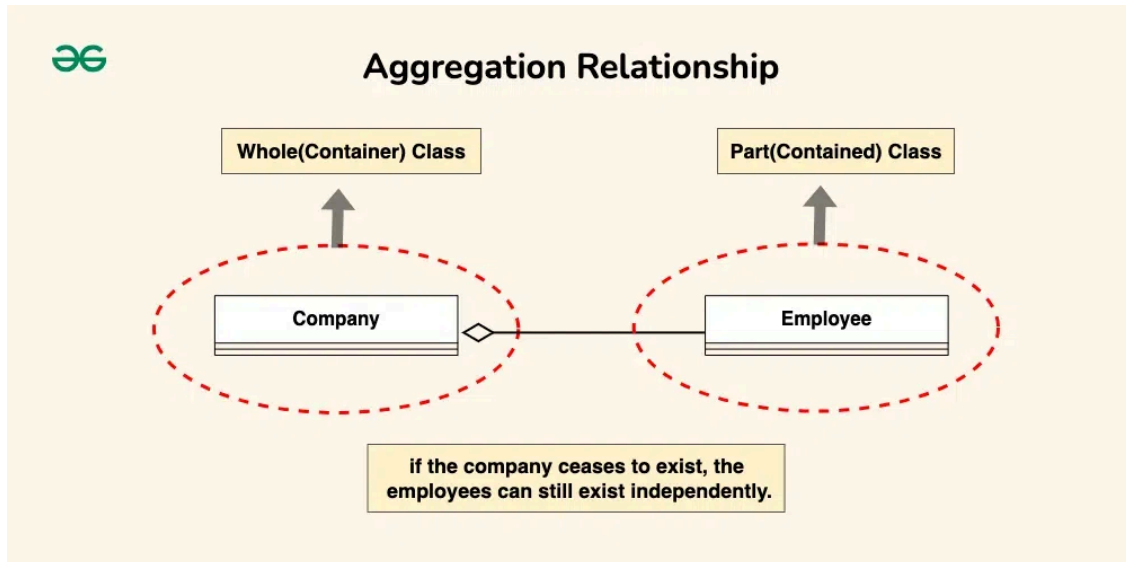


3. Aggregation

Aggregation is a specialized form of association that represents a “whole-part” relationship. It denotes a stronger relationship where one class (the whole) contains or is composed of another class (the part). Aggregation is represented by a diamond shape on the side of the whole class. In this kind of relationship, the child class can exist independently of its parent class.

Let's understand aggregation using an example:

The company can be considered as the whole, while the employees are the parts.
Employees belong to the company, and the company can have multiple employees.
However, if the company ceases to exist, the employees can still exist independently.



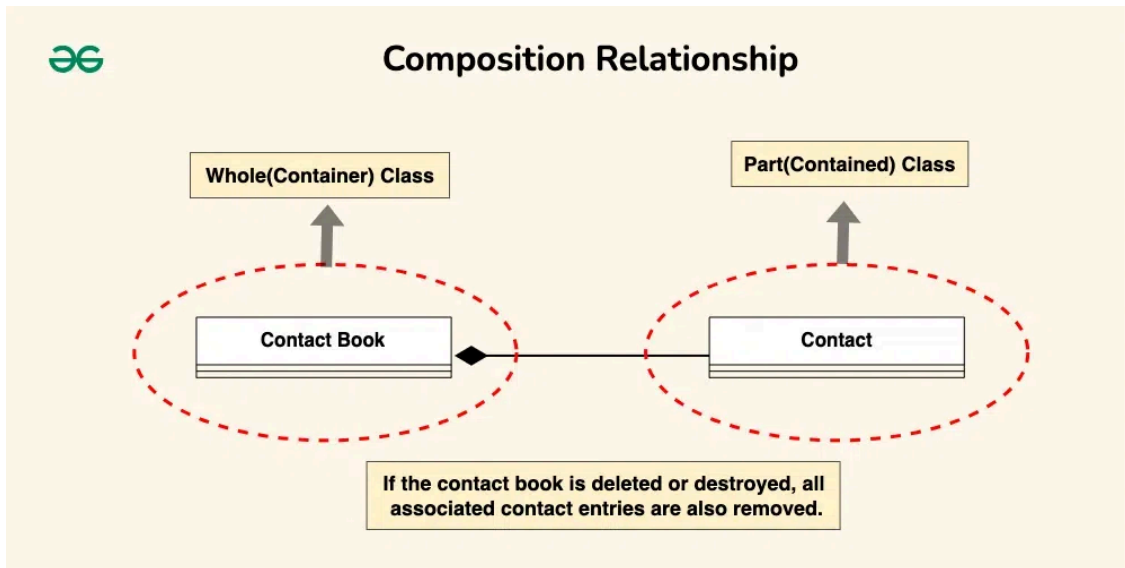
4. Composition

Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship. In composition, the part class cannot exist independently of the whole class. Composition is represented by a filled diamond shape on the side of the whole class.

Let's understand Composition using an example:

Imagine a digital contact book application. The contact book is the whole, and each contact entry is a part. Each contact entry is fully owned and managed by the contact book. If the contact book is deleted or destroyed, all associated contact entries are also removed.

This illustrates composition because the existence of the contact entries depends entirely on the presence of the contact book. Without the contact book, the individual contact entries lose their meaning and cannot exist on their own.

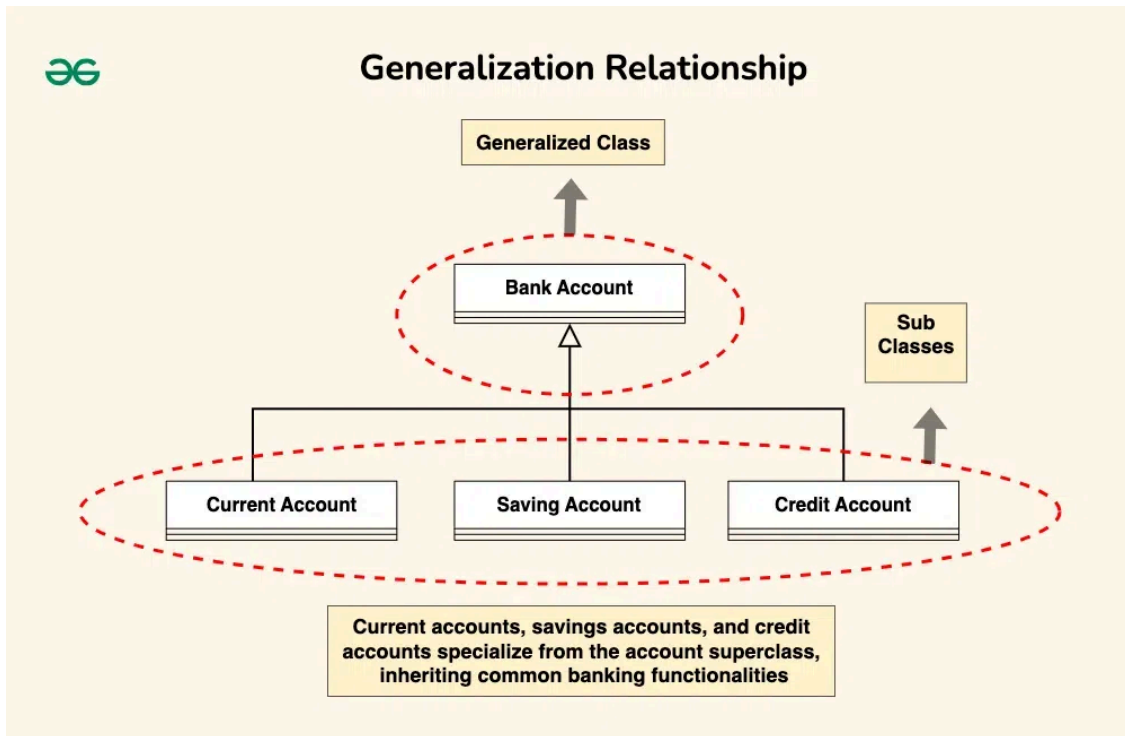


5. Generalization(Inheritance)

Inheritance represents an “is-a” relationship between classes, where one class (the subclass or child) inherits the properties and behaviors of another class (the superclass or parent). Inheritance is depicted by a solid line with a closed, hollow arrowhead pointing from the subclass to the superclass.

In the example of bank accounts, we can use generalization to represent different types of accounts such as current accounts, savings accounts, and credit accounts.

The Bank Account class serves as the generalized representation of all types of bank accounts, while the subclasses (Current Account, Savings Account, Credit Account) represent specialized versions that inherit and extend the functionality of the base class.



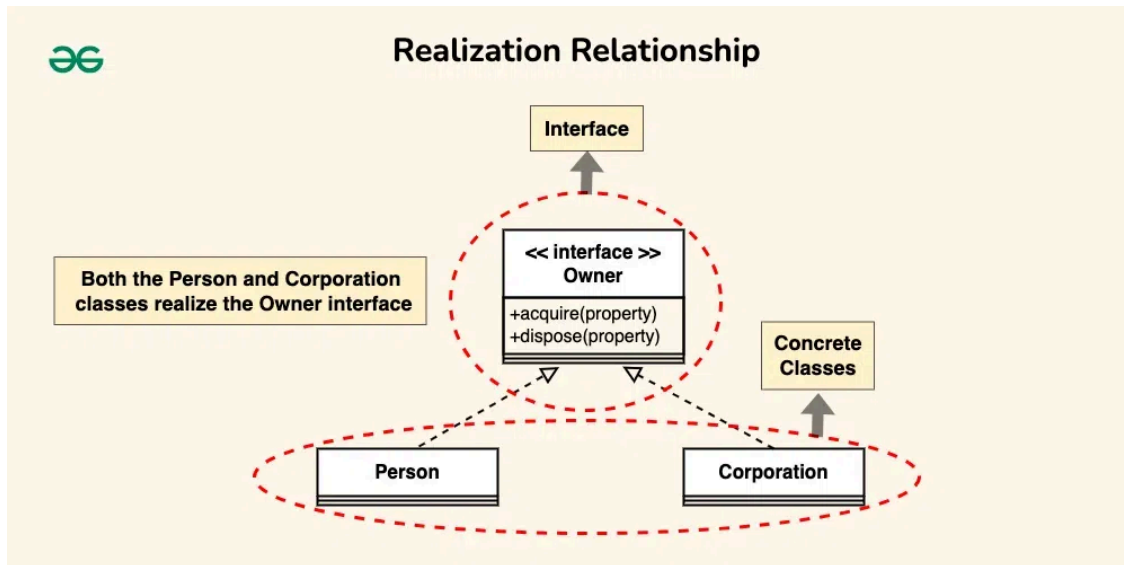
6. Realization (Interface Implementation)

Realization indicates that a class implements the features of an interface. It is often used in cases where a class realizes the operations defined by an interface. Realization is depicted by a dashed line with an open arrowhead pointing from the implementing class to the interface.

Let's consider the scenario where a "Person" and a "Corporation" both realizing an "Owner" interface.

- **Owner Interface:** This interface now includes methods such as "acquire(property)" and "dispose(property)" to represent actions related to acquiring and disposing of property.
- **Person Class (Realization):** The Person class implements the Owner interface, providing concrete implementations for the "acquire(property)" and "dispose(property)" methods. For instance, a person can acquire ownership of a house or dispose of a car.
- **Corporation Class (Realization):** Similarly, the Corporation class also implements the Owner interface, offering specific implementations for the "acquire(property)" and "dispose(property)" methods. For example, a corporation can acquire ownership of real estate properties or dispose of company vehicles.

Both the Person and Corporation classes realize the Owner interface, meaning they provide concrete implementations for the “acquire(property)” and “dispose(property)” methods defined in the interface.



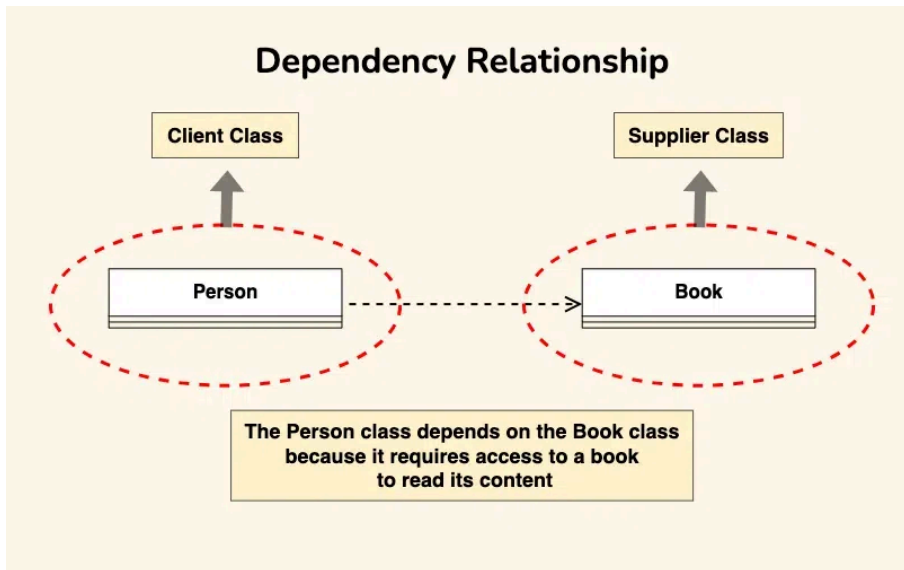
7. Dependency Relationship

A dependency exists between two classes when one class relies on another, but the relationship is not as strong as association or inheritance. It represents a more loosely coupled connection between classes. Dependencies are often depicted as a dashed arrow.

Let's consider a scenario where a Person depends on a Book.

- **Person Class:** Represents an individual who reads a book. The Person class depends on the Book class to access and read the content.
- **Book Class:** Represents a book that contains content to be read by a person. The Book class is independent and can exist without the Person class.

The Person class depends on the Book class because it requires access to a book to read its content. However, the Book class does not depend on the Person class; it can exist independently and does not rely on the Person class for its functionality.



8. Usage(Dependency) Relationship

A usage dependency relationship in a UML class diagram indicates that one class (the client) utilizes or depends on another class (the supplier) to perform certain tasks or access certain functionality. The client class relies on the services provided by the supplier class but does not own or create instances of it.

- Usage dependencies represent a form of dependency where one class depends on another class to fulfill a specific need or requirement.
- The client class requires access to specific features or services provided by the supplier class.
- In UML class diagrams, usage dependencies are typically represented by a dashed arrowed line pointing from the client class to the supplier class.
- The arrow indicates the direction of the dependency, showing that the client class depends on the services provided by the supplier class.

Consider a scenario where a “Car” class depends on a “FuelTank” class to manage fuel consumption.

- The “Car” class may need to access methods or attributes of the “FuelTank” class to check the fuel level, refill fuel, or monitor fuel consumption.
- In this case, the “Car” class has a usage dependency on the “FuelTank” class because it utilizes its services to perform certain tasks related to fuel management.

